

Caderno de Exercicios

Python — Do Zero a Arquitetura

70+ exercicios praticos organizados nas 4 fases da formacao:
Base • Maestria • Qualidade • Arquitetura

Material complementar de estudo

Sumario

Fase 0 — Base

- enumerate
- list comprehension e operador ternario
- json integration
- sys module
- with (context manager)
- manipulacao de arquivos
- yield
- walrus operator (:=)

Fase 1 — Maestria da Linguagem e Web

- *args e **kwargs
- decorators e closures
- lambda / filter / map / reduce
- dunder methods
- zip
- generators e iterators
- nested / dict comprehension
- typing e type hints
- regex
- async e await
- SQL integration e ORM

Fase 2 — Qualidade, Validacao e CI/CD

- pydantic
- pytest
- black e formatacao
- lint e ruff
- logging
- pdb e ipdb

Fase 3 — Arquitetura e Alta Escala

- decorators e memoization
- staticmethod / classmethod
- function composition e currying
- threading

Bonus — Topicos de Nicho

- argparse
- mypy
- fuzzy matching

Fase 0 — Base (Pensamento Computacional)

enumerate

Exercicio 01

Dada a lista `frutas = ['manga', 'banana', 'acerola', 'uva']`, use `enumerate` para imprimir cada fruta precedida de seu indice (ex.: `0 - manga`).

Exercicio 02

Voce recebeu uma lista de temperaturas diarias (em Celsius). Use `enumerate` para encontrar e imprimir o dia (indice + 1) e o valor de todas as temperaturas acima de 30 graus.

```
temps = [28, 31, 27, 35, 29, 33, 26]
```

Dica: enumerate aceita um argumento start=1.

Exercicio 03

Escreva uma funcao `buscar(lista, valor)` que retorne o indice da primeira ocorrencia de *valor* na lista usando `enumerate`. Se nao existir, retorne -1.

List Comprehension e Operador Ternario

Exercicio 04

A partir da lista `numeros = list(range(1, 21))`, crie uma nova lista contendo apenas os numeros pares usando list comprehension.

Exercicio 05

Dada uma lista de palavras, gere uma nova lista com cada palavra em MAIUSCULO se ela tiver mais de 5 letras, ou em minusculo caso contrario. Use operador ternario dentro de uma list comprehension.

```
palavras = ["Python", "AI", "Desenvolvimento", "API", "Web", "Framework"]
```

Exercicio 06

Crie uma list comprehension que gere uma lista de tuplas (n, n^2) para n de 1 a 15, mas apenas quando n^2 for impar.

JSON Integration

Exercicio 07

Crie um dicionario com dados de um produto (nome, preco, estoque). Salve-o em um arquivo `produto.json` e, em seguida, leia o arquivo de volta para outro dicionario e imprima o preco.

```
import json
```

Exercicio 08

Voce recebeu o seguinte JSON como string:

```
'{"alunos": [{"nome": "Ana", "nota": 8.5}, {"nome": "Pedro", "nota": 6.0}]}'
```

Parse a string, calcule a media das notas e adicione o campo `"media"` ao dicionario. Salve o resultado em `turma.json`.

sys Module

Exercicio 09

Escreva um script **saudacao.py** que receba um nome via **sys.argv** e imprima 'Ola, <nome>!'. Se nenhum argumento for passado, imprima uma mensagem de uso e encerre com **sys.exit(1)**.

```
import sys
```

Exercicio 10

Crie um script que receba dois numeros como argumentos de linha de comando e imprima a soma. Trate o caso de o usuario passar valores nao numericos usando try/except.

```
# Execucao: python soma.py 10 25
```

with (Context Manager)

Exercicio 11

Abra um arquivo **notas.txt** usando **with open(...)**, escreva 3 linhas com nomes e notas e, em seguida, reabra o arquivo para leitura (tambem com with) e imprima o conteudo. Explique por que o with dispensa o **close()** manual.

```
with open("notas.txt", "w") as f: f.write("Ana 8.5\n")
```

Exercicio 12

Crie seu proprio context manager usando uma classe com os metodos **__enter__** e **__exit__**. A classe **Timer** deve medir o tempo gasto dentro do bloco with e imprimi-lo ao sair.

```
import time with Timer() as t: sum(range(1_000_000)) # Saida: Tempo: 0.032s
```

Exercicio 13

Reescreva o context manager Timer do exercicio anterior usando **@contextlib.contextmanager** com **yield** em vez de uma classe completa.

```
from contextlib import contextmanager
```

Dica: O yield separa o codigo de entrada (__enter__) do de saida (__exit__).

Manipulacao de Arquivos

Exercicio 14

Escreva um script que crie um arquivo **alunos.csv** com colunas nome, nota1, nota2. Insira pelo menos 5 registros. Depois, leia o arquivo e imprima apenas os alunos cuja media das duas notas seja ≥ 7 .

```
# Use open() para escrita e leitura, split(',') para parsear
```

Exercicio 15

Crie uma funcao **contar_palavras(caminho: str) -> dict** que leia um arquivo de texto e retorne um dicionario com a frequencia de cada palavra (ignorando maiusculas/minusculas).

Dica: Use collections.Counter para simplificar.

Exercicio 16

Escreva um script que percorra todos os arquivos **.txt** de um diretorio usando **pathlib.Path**, imprima o nome e o tamanho em KB de cada um, e copie aqueles maiores que 1 KB para uma

subpasta **grandes/**.

```
from pathlib import Path import shutil
```

Exercicio 17

Implemente uma funcao **substituir_em_arquivo(caminho, antiga, nova)** que leia um arquivo, substitua todas as ocorrencias de *antiga* por *nova* e salve o resultado. Use um arquivo temporario para evitar perda de dados se o script falhar no meio.

```
import tempfile, os
```

Dica: Escreva no tempfile, depois use os.replace() para mover atomicamente.

yield

Exercicio 18

Escreva um generator **pares_infinitos()** que produza numeros pares infinitamente (2, 4, 6, ...). Use **yield** e consuma apenas os 10 primeiros valores com um loop e **break** ou com **itertools.islice**.

```
from itertools import islice
```

Exercicio 19

Crie um generator **ler_linhas(caminho)** que leia um arquivo linha a linha usando yield (sem carregar tudo na memoria). Use-o para imprimir apenas as linhas que contenham a palavra 'erro' (case-insensitive).

Dica: Combine with open() com yield dentro do generator.

Exercicio 20

Implemente um pipeline de generators: **numeros()** gera inteiros de 1 a 100, **quadrados(seq)** gera o quadrado de cada item, e **filtrar_grandes(seq, limite)** filtra os maiores que *limite*. Encadeie os tres e consuma o resultado.

```
# Composicao: filtrar_grandes(quadrados(numeros()), 500)
```

Walrus Operator (:=)

Exercicio 21

Use o walrus operator para simplificar o codigo abaixo em uma unica condicao, eliminando a variavel intermediaria:

```
n = len(lista)
```

```
if n > 10:
```

```
    print(f'Lista grande: {n} itens')
```

```
# Resultado esperado: # if (n := len(lista)) > 10: # print(f'Lista grande: {n} itens')
```

Exercicio 22

Escreva um loop que leia linhas de entrada do usuario com **input()** ate que ele digite 'sair'. Use o walrus operator para atribuir e testar na mesma expressao do **while**.

```
# while (linha := input(">>> ")) != "sair":
```

Exercicio 23

Dada uma lista de strings, use uma list comprehension com walrus operator para criar uma lista de tuplas (palavra, tamanho) apenas para palavras cujo tamanho seja maior que 4.

```
palavras = ["oi", "Python", "dev", "asyncio", "API", "framework"] # [(p,  
tam) for p in palavras if (tam := len(p)) > 4]
```

Fase 1 — Maestria da Linguagem e Web

*args e **kwargs

Exercicio 24

Escreva uma funcao **media(*args)** que aceite um numero variavel de notas e retorne a media aritmetica. Trate o caso de nenhum argumento.

```
print(media(7, 8.5, 9, 6)) # 7.625
```

Exercicio 25

Crie uma funcao **criar_perfil(nome, **kwargs)** que retorne um dicionario com o nome e qualquer informacao extra passada como keyword argument.

```
criar_perfil("Ana", idade=25, cidade="SP") #  
{ "nome": "Ana", "idade": 25, "cidade": "SP" }
```

Exercicio 26

Implemente uma funcao **aplicar(func, *args, **kwargs)** que receba uma funcao qualquer e a execute com os argumentos fornecidos, retornando o resultado.

Decorators e Closures

Exercicio 27

Crie um decorator **@cronometro** que meça e imprima o tempo de execucao de qualquer funcao decorada. Use-o em uma funcao que some numeros de 1 a 1_000_000.

```
import time
```

Exercicio 28

Escreva uma closure **multiplicador(fator)** que retorne uma funcao interna capaz de multiplicar qualquer numero pelo fator capturado.

```
dobro = multiplicador(2) print(dobro(5)) # 10
```

Exercicio 29

Crie um decorator **@requer_login(usuario)** que receba o nome de um usuario e, se ele for 'admin', execute a funcao decorada; caso contrario, imprima 'Acesso negado'.

Dica: Decorator com argumento requer tres niveis de funcao aninhada.

Lambda, filter, map e reduce

Exercicio 30

Dada a lista **precos = [120, 45, 200, 89, 15, 300]**, use **filter** com **lambda** para obter apenas precos acima de 100.

Exercicio 31

Use **map** com **lambda** para converter uma lista de temperaturas em Fahrenheit para Celsius.
fahrenheit = [32, 68, 100, 212]

*Dica: $C = (F - 32) * 5/9$*

Exercicio 32

Usando **reduce** do modulo **functools**, calcule o produto de todos os elementos de uma lista de inteiros.

```
from functools import reduce
```

Dunder Methods

Exercicio 33

Crie uma classe **Produto** com atributos **nome** e **preco**. Implemente **__str__** (exibicao amigavel) e **__repr__** (representacao tecnica) de forma que:

str(p) retorne 'Produto: Camiseta - R\$ 59.90'

repr(p) retorne 'Produto("Camiseta", 59.90)'

Exercicio 34

Implemente os metodos **__eq__** e **__lt__** na classe **Produto** para que duas instancias possam ser comparadas pelo preco. Depois, ordene uma lista de produtos com **sorted()**.

Exercicio 35

Crie uma classe **Carrinho** que armazene itens internamente e implemente **__len__** (numero de itens), **__contains__** (verificar se item existe) e **__getitem__** (acessar por indice).

zip

Exercicio 36

Dadas duas listas — **nomes = ['Ana', 'Bruno', 'Carla']** e **notas = [8.5, 7.0, 9.2]** — use **zip** para criar um dicionario {nome: nota}.

Exercicio 37

Voce tem tres listas paralelas: **produtos**, **precos** e **estoques**. Use **zip** para gerar uma lista de dicionarios onde cada dicionario tem chaves 'produto', 'preco' e 'estoque'.

```
produtos = ["Mouse", "Teclado", "Monitor"]  
precos = [79.9, 149.9, 899.9]  
estoques = [150, 80, 25]
```

Generators e Iterators

Exercicio 38

Escreva um generator **fibonacci(n)** que produza os primeiros **n** numeros da sequencia de Fibonacci usando **yield**.

```
for num in fibonacci(10): print(num)
```

Exercicio 39

Crie um generator **ler_em_blocos(arquivo, tamanho=1024)** que leia um arquivo de texto em blocos de *tamanho* bytes, simulando o processamento de um arquivo grande sem carregar tudo na memoria.

Dica: Use open() com read(tamanho) em um loop while.

Exercicio 40

Implemente o protocolo iterator criando uma classe **Contador** com `__iter__` e `__next__` que conte de um valor inicial ate um valor final.

Nested e Dict Comprehension

Exercicio 41

Crie um dicionario {aluno: 'aprovado'/'reprovado'} a partir de:

resultados = {'Ana': 7.5, 'Pedro': 4.0, 'Maria': 9.0, 'Joao': 5.5}

usando dict comprehension (aprovado se nota >= 6).

Exercicio 42

Dada uma matriz 3x3 representada como lista de listas, use nested comprehension para gerar a transposta.

```
matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Dica: `transposta[j][i] = matriz[i][j]`

Typing e Type Hints

Exercicio 43

Reescreva a funcao abaixo adicionando type hints completos nos parametros e no retorno:

def filtrar_maiores(lista, limite):

return [x for x in lista if x > limite]

```
from typing import List
```

Exercicio 44

Crie uma funcao **buscar_usuario(user_id: int) -> dict | None** que simule uma busca em banco de dados. Use **Optional** ou a sintaxe union **X | Y** no retorno.

Regex (Expressoes Regulares)

Exercicio 45

Escreva uma funcao **validar_email(email: str) -> bool** que use **re.match** para verificar se um e-mail tem formato basico valido (algo@dominio.ext).

```
import re
```

Exercicio 46

Dada uma string com texto livre, extraia todos os valores monetarios no formato 'R\$ X.XXX,XX' ou 'R\$ XXX,XX' usando **re.findall**.

```
texto = "O produto custa R$ 1.299,90 e o frete R$ 45,00"
```

Exercicio 47

Crie uma funcao **limpar_cpf(cpf: str) -> str** que receba um CPF com pontos e tracos (ex.: '123.456.789-00') e retorne apenas os digitos usando **re.sub**.

Async e Await

Exercicio 48

Usando **asyncio**, crie duas coroutines — **tarefa_a()** e **tarefa_b()** — que simulem operacoes de I/O com **await asyncio.sleep()**. Execute ambas concorrentemente com **asyncio.gather** e mostre que o tempo total e menor que a soma.

```
import asyncio
```

Exercicio 49

Escreva uma funcao assincrona **buscar_dados(urls: list[str])** que simule requisicoes HTTP concorrentes (use sleep como placeholder) e retorne uma lista de resultados na mesma ordem das URLs.

Dica: Use `asyncio.gather(*[fetch(url) for url in urls])`.

SQL Integration e ORM

Exercicio 50

Usando o modulo **sqlite3**, crie um banco de dados em memoria, uma tabela **produtos(id, nome, preco)**, insira 3 registros e faca um SELECT para imprimir todos os produtos com preco > 50.

```
import sqlite3 conn = sqlite3.connect(':memory:')
```

Exercicio 51

Refatore o exercicio anterior usando **SQLAlchemy Core**: defina a tabela com **Table()** e **MetaData()**, insira os mesmos registros e execute a consulta equivalente.

```
from sqlalchemy import create_engine, MetaData, Table, Column, Integer, String, Float
```

Fase 2 — Qualidade, Validacao e CI/CD

Pydantic

Exercicio 52

Crie um modelo Pydantic **Usuario** com campos: nome (str), email (str) e idade (int, minimo 0). Instancie com dados corretos e depois com dados invalidos para observar o erro de validacao.

```
from pydantic import BaseModel, Field
```

Exercicio 53

Crie um modelo **Pedido** que contenha uma lista de **Item** (outro modelo com nome e preco). Adicione um **@field_validator** que garanta que o preco de cada item seja positivo.

Dica: Use `model_validate()` para testar com JSON.

Pytest

Exercicio 54

Escreva uma funcao **calcular_desconto(preco, percentual)** e crie no minimo 3 testes com **pytest**: um para desconto normal, um para percentual zero e um que verifique se levanta **ValueError** para percentual negativo.

```
# test_desconto.py import pytest
```

Exercicio 55

Usando **@pytest.fixture**, crie uma fixture que retorne uma lista de produtos de teste. Use essa fixture em dois testes diferentes: um que verifique o total e outro que verifique a quantidade de itens.

Black e Formatacao (PEP 8)

Exercicio 56

Dado o codigo abaixo (propositalmente mal formatado), reescreva-o manualmente seguindo a PEP 8 e depois confira com **black --check**:

```
def soma( a,b ): return a+b
x={'chave' : 'valor','outra':'abc'}
lista=[1,2, 3 , 4]
```

Lint e Ruff

Exercicio 57

Crie um arquivo Python com pelo menos 3 violacoes propositalis (import nao usado, variavel nao utilizada, linha > 120 caracteres). Execute **ruff check** e corrija cada violacao reportada.

```
# pip install ruff # ruff check meu_script.py
```

Logging

Exercicio 58

Substitua todos os **print()** do código abaixo por chamadas equivalentes do módulo **logging** (debug, info, warning, error):

```
print('Iniciando...')
print('Usuario nao encontrado')
print('ERRO: conexao falhou')
```

```
import logging
logging.basicConfig(level=logging.DEBUG)
```

Exercício 59

Configure o logging para gravar mensagens em um arquivo **app.log** com formato: *timestamp - nivel - mensagem*. Gere mensagens INFO e ERROR e verifique o arquivo.

Dica: Use logging.FileHandler e logging.Formatter.

pdb e ipdb (Debugging)

Exercício 60

Insira um **breakpoint()** dentro da função abaixo e use os comandos **n** (next), **p** (print) e **c** (continue) para inspecionar o valor de *total* a cada iteração:

```
def soma_lista(nums):
    total = 0
    for n in nums:
        total += n
    return total
```

```
# Python 3.7+: breakpoint() equivale a import pdb; pdb.set_trace()
```

Fase 3 — Arquitetura e Alta Escala

Decorators e Memoization

Exercicio 61

Implemente um decorator **@memoize** do zero (sem usar `functools`) que armazene resultados em um dicionario. Aplique-o a uma funcao recursiva de Fibonacci e compare a velocidade com e sem cache.

Dica: O dicionario de cache pode ficar na closure do decorator.

Exercicio 62

Refatore o exercicio anterior usando **@functools.lru_cache**. Use **cache_info()** para imprimir estatisticas de hits e misses.

```
from functools import lru_cache
```

@staticmethod e @classmethod

Exercicio 63

Crie uma classe **Conversor** com um **@staticmethod celsius_para_fahrenheit(c)** e um **@classmethod from_config(cls, config_dict)** que crie uma instancia a partir de um dicionario de configuracao.

Exercicio 64

Implemente um Factory Method: crie uma classe **Notificacao** com subclasses **Email** e **SMS**. Use um **@classmethod** na classe base como fabrica que retorne a subclasse correta baseado em um parametro 'tipo'.

Dica: Padrao de projeto Factory Method.

Function Composition e Currying

Exercicio 65

Escreva uma funcao **compose(*funcs)** que receba N funcoes e retorne uma nova funcao que as aplique da direita para a esquerda.

Exemplo: `compose(str.upper, str.strip)(' ola ') -> 'OLA'`

```
from functools import reduce
```

Exercicio 66

Implemente uma versao 'curried' de uma funcao de desconto:

aplicar_desconto(percentual)(preco) -> preco com desconto.

Use essa funcao para criar funcoes especializadas como **desconto_10 = aplicar_desconto(0.10)**.

Threading (Computacao Paralela)

Exercicio 67

Crie duas funcoes que simulem tarefas pesadas (com `time.sleep`). Execute-as sequencialmente e depois em paralelo usando **threading.Thread**. Compare e imprima o tempo total de cada

abordagem.

```
import threading, time
```

Exercício 68

Use **concurrent.futures.ThreadPoolExecutor** para baixar (simular com sleep) 5 'paginas' em paralelo. Mostre o resultado de cada 'download' conforme ele termina usando **as_completed**.

```
from concurrent.futures import ThreadPoolExecutor, as_completed
```

Bonus — Topicos de Nicho

argparse

Exercicio 69

Crie um script CLI com **argparse** que aceite os argumentos **--nome** (obrigatorio), **--saudacao** (opcional, padrao='Ola') e imprima a saudacao personalizada.

```
import argparse parser = argparse.ArgumentParser(description='Saudacao CLI')
```

Exercicio 70

Estenda o script anterior adicionando um subcomando **listar** e outro **criar**, simulando um mini gerenciador de tarefas via linha de comando.

Dica: Use `parser.add_subparsers()`.

mypy (Validacao Estatica de Tipos)

Exercicio 71

Dado o codigo abaixo, adicione type hints e execute **mypy** para encontrar o erro de tipo:

```
def dobro(x):  
    return x * 2
```

```
resultado = dobro('cinco') + 10
```

```
# pip install mypy # mypy script.py
```

Fuzzy Matching

Exercicio 72

Usando a biblioteca **thefuzz**, crie uma funcao **buscar_produto(query, catalogo)** que retorne o produto do catalogo com maior similaridade ao texto digitado pelo usuario. Use um limiar minimo de 60% de similaridade.

```
from thefuzz import fuzz, process catalogo = ["Camiseta Preta", "Calca  
Jeans", "Tenis Branco", "Bone Azul"]
```


Fim do Caderno

Pratique cada exercicio no seu editor favorito.
Releia o enunciado, tente resolver sem olhar dicas,
e so depois confira com a documentacao oficial do Python.

Bons estudos!